Epic Go-Live & Command Center Endpoint Readiness Playbook

Implementation Solution Guide

Version 1.0.0 · 2026-06-27

Contents

Executive Summary — Epic Go-Live & Command Center Endpoint Readiness Playbook 4

The challenge . . . 4

The solution . . . 4

What makes it different . . . 4

What's included . . . 5

Business outcome . . . 5

1. Solution Overview — Epic Go-Live & Command Center Endpoint Readiness Playbook 5

What this solution is . . . 5

The problem it solves . . . 6

Who this is for . . . 6

Scope . . . 6

Approach — the phase model . . . 7

Named artifacts this playbook produces . . . 7

What you get . . . 8

Related solutions (deep config lives here) . . . 8

2. Requirements — Epic Go-Live & Command Center Endpoint Readiness Playbook 8

R1 — Authoritative in-scope device inventory . . . 9

R2 — Defined, measurable device readiness standard . . . 9

R3 — Correct Epic client version present on every device . . . 9

R4 — Functional clinical printing . . . 10

R5 — Attached peripherals present and working . . . 10

R6 — Imprivata tap-and-go (SSO) functional . . . 11

R7 — Login time within target . . . 11

R8 — Devices enrolled and compliant . . . 11

R9 — Command-center operating model defined . . . 12

R10 — Readiness device group for targeting . . . 12

R11 — Go/no-go gate and decision authority . . . 12

R12 — Hypercare exit criteria defined . . . 13

3. Implementation — Epic Go-Live & Command Center Endpoint Readiness Playbook 13

Phase 1 — Assess . . . 13

Phase 2 — Prepare / Remediate . . . 14

Phase 3 — Validate (go/no-go) . . . 16

Phase 4 — Command Center (hypercare) . . . 16

Phase 5 — Stabilize / Handover . . . 18

5. Testing & Validation — Epic Go-Live & Command Center Endpoint Readiness Playbook 19

1. The validation sweep (per-device PASS/FAIL) . . . 19
2. Readiness scorecard (aggregate evidence) . . . 20
3. Functional spot-tests (human-in-the-loop) . . . 20
4. Test-device rehearsal of remediation (safe-change proof) . . . 20
5. Command-center operational rehearsal (dry run) . . . 21
6. Go/no-go gate validation . . . 21
7. Exit-criteria validation (hypercare → steady state) . . . 21
Validation checklist (summary) . . . 21
6. Troubleshooting — Epic Go-Live & Command Center Endpoint Readiness Playbook 22
A. Login / SSO — badge tap doesn't log in . . . 22
B. Printing — wristband / label / document won't print . . . 23
C. Enrollment / compliance — device can't reach Epic or its dependencies . . . 23
D. Performance — slow login or slow session . . . 24
E. Wrong / missing Epic client version . . . 24
F. Peripheral not working — scanner / badge reader / signature pad . . . 25
G. Tooling / script issues . . . 25
H. Command-center / process issues . . . 26
7. Best Practices — Epic Go-Live & Command Center Endpoint Readiness Playbook 26
Readiness standard and validation . . . 26
Targeting and change safety . . . 27
Command center . . . 27
Comms and escalation . . . 27
Compliance and governance . . . 28
Handover and reuse . . . 28
8. Deliverables Summary — Epic Go-Live & Command Center Endpoint Readiness Playbook 28
Documentation . . . 29
PowerShell scripts . . . 30
Visual asset . . . 31
Named artifacts the playbook produces (operating outputs) . . . 31
How the pieces fit . . . 31
Related solutions (deep configuration) . . . 32
Appendix A — PowerShell Scripts 33

Executive Summary — Epic Go-Live & Command Center Endpoint Readiness Playbook

The challenge

An Epic go-live — a new install, a major upgrade, or a new-site activation — is one of the highest-stakes events a healthcare organization undertakes. On day 1, clinical staff are learning new software while depending entirely on the device in front of them to deliver care. The endpoint layer — workstations, workstations-on-wheels, exam-room PCs, shared kiosks, and the printers, scanners, and badge readers attached to them — is simultaneously the **most common source of go-live problems** and frequently the **least prepared-for**. When a wristband won't print, a badge tap won't log a nurse in, a device was never enrolled, or a workstation is too slow to chart, the failure lands at the bedside, at the busiest moment, in front of a patient.

The solution

This playbook makes the clinical endpoint fleet **provably ready** for the activation and **runs the command center** through hypercare until the environment stabilizes. It delivers a complete operating model in five phases:

- **Assess** — build a trusted inventory of every in-scope device and snapshot its current readiness.
- **Prepare / Remediate** — close every gap: enrollment, compliance, the correct Epic client version, printing, peripherals, Imprivata tap-and-go, and login times.
- **Validate (go/no-go)** — a per-device PASS/FAIL sweep rolled into a readiness scorecard that turns thousands of device results into a single, defensible go/no-go recommendation.
- **Command Center (hypercare)** — a defined structure (roles, tiers, dispatch, at-the-elbow support), a rapid-fix triage runbook for the predictable day-1 failures, and clear comms and escalation.
- **Stabilize / Handover** — measurable exit criteria that retire the command center and hand a documented environment to the steady-state service desk.

What makes it different

- **Readiness is measurable and machine-proven.** Every criterion is a concrete threshold tested by automation, so "ready" means the same thing to everyone and the go/no-go recommendation is backed by evidence, not opinion.
- **Built for clinical speed and safety.** The command-center model puts support at-the-elbow on the units, fixes the clinician's problem *now* (manual fallback, ready spares) and repairs the device later, and treats safety-critical failures (wristband print, medication-administration tap) as a separate, non-negotiable gate.
- **Safe by default.** Discovery, validation, and reporting are read-only; all remediation defaults to a preview mode and changes nothing until explicitly applied, inside an approved window, after a test-device run — fully aligned with HIPAA/HITRUST change control.

- **Reusable.** Every artifact — readiness checklist, go/no-go gate, command-center roster, triage runbook, scorecard — is white-label and carries forward to the next wave, site, or upgrade, so each activation starts ahead of the last.

What's included

Eight documents plus an executive summary, **five PowerShell automation scripts** (discovery, validation, reporting, remediation, and readiness-group management via Microsoft Intune / Entra and Microsoft Graph), and a hero image — a turnkey package an endpoint team can run end-to-end.

Business outcome

Fewer day-1 endpoint failures, faster resolution of the ones that occur, a defensible go/no-go decision for leadership, and a clean transition to steady state — protecting clinician productivity and, ultimately, patient care during the most disruptive week in the organization's IT calendar.

1. Solution Overview — Epic Go-Live & Command Center Endpoint Readiness Playbook

What this solution is

This playbook turns the clinical endpoint fleet into a **go-live-ready state** for an Epic activation — whether a brand-new Epic install, a major upgrade, or a new-site / new-facility activation — and then **runs the command center** through hypercare until the environment stabilizes into normal operations. It is a repeatable operational playbook, not a one-time runbook: every artifact it defines is reusable for the next wave, the next hospital, or the next upgrade.

The solution covers the full arc an endpoint team owns around an Epic go-live:

- **Pre-go-live device readiness** — a concrete, measurable readiness standard for every clinical workstation, WOW (workstation-on-wheels), exam-room PC, shared kiosk, and clinical peripheral, plus a **validation sweep** that proves each device meets it (correct Epic client version, working printing, attached/working peripherals, Imprivata tap-and-go, acceptable login times, current compliance posture).
- **A go/no-go decision gate** — an evidence-backed checklist that converts thousands of individual device results into a single defensible go/no-go recommendation for the Epic program leadership.
- **Command-center operations** — the structure, roles, tiers, dispatch model, and at-the-elbow support pattern that staffs the activation and triages issues at the speed a live clinical floor demands.
- **A rapid-fix triage runbook** — step-by-step fixes for the handful of endpoint problems that cause the overwhelming majority of day-1 tickets: login/SSO failures, printing failures, not-enrolled / non-compliant devices, and slow performance.
- **Comms and escalation** — who is told what, when, and how an endpoint issue escalates from at-the-elbow to command center to the Epic application or infrastructure teams.
- **Hypercare-to-steady-state handover** — the criteria and the handoff that retire the command center and move ownership to the standing service desk.

*This playbook is **endpoint-focused**. It deliberately does not re-document deep Epic client configuration, Imprivata appliance setup, or workstation image standardization — those live in their own dedicated solutions (see *Related solutions* below) and are referenced wherever a deep dive is needed.*

The problem it solves

An Epic go-live is one of the highest-stakes events a healthcare organization runs. Clinical staff are simultaneously learning new software *and* depending on the device in front of them to do their jobs at the bedside. When the endpoint fails — the printer won't print a wristband, the badge tap won't log the nurse in, the workstation is too slow to chart, or the device was never enrolled and can't reach Epic — the failure lands at the worst possible moment, on the busiest possible day, in front of a patient.

Endpoint problems are also the **most common and most visible** category of go-live tickets, yet they are frequently the **least prepared-for**. Application training gets months; the devices often get a rushed, undocumented check the week before. Without a defined readiness standard, a validation sweep that proves it, and a command center built to triage at clinical speed, the endpoint layer becomes the bottleneck that drags down an otherwise well-run activation.

This solution closes that gap. It gives the endpoint team a defensible standard, the tooling to prove the fleet meets it, a command-center operating model staffed for the floor, and a triage runbook that resolves the predictable day-1 failures in minutes instead of hours.

Who this is for

- **Endpoint / EUC (end-user computing) and desktop engineering teams** who own the clinical device fleet and are accountable for its readiness at go-live.
- **Epic go-live / activation program leadership** who need an evidence-backed go/no-go recommendation for the endpoint domain.
- **Command-center leads and dispatchers** who run the activation floor and need a defined structure, roster, and triage runbook.
- **At-the-elbow (ATE) support staff and floor walkers** who need a fast, consistent way to fix the common endpoint failures and to escalate the ones they can't.
- **Service desk and operations leadership** who will inherit the environment at hypercare exit and need a clean handover.

Scope

In scope

- Defining and applying a **device readiness standard** for clinical endpoints (workstations, WOWs, exam-room PCs, shared/kiosk devices, and attached clinical peripherals).
- A **validation sweep** that proves readiness per device: Epic client version present and correct, printing functional, peripherals attached and working, Imprivata tap-and-go functional, login time within target, device enrolled and compliant.
- A **go/no-go gate** that aggregates per-device results into a readiness scorecard and a go/no-go recommendation.

- **Command-center structure**: roles, support tiers, the dispatch model, and the at-the-elbow support pattern.
- A **rapid-fix triage runbook** for the highest-volume day-1 endpoint problems.
- **Comms and escalation** paths during activation.
- **Hypercare-to-steady-state handover** criteria and handoff.
- Five PowerShell helper scripts (discovery, validation, reporting, remediation, and readiness-group management) that automate the read-only evidence-gathering and the safe remediation actions this playbook depends on.

Out of scope (referenced, not re-documented)

- Deep Epic Hyperdrive / Hyperspace client packaging, configuration, and deployment internals.
- Imprivata OneSign appliance, agent, and policy configuration internals.
- Master workstation image build and standardization internals.
- Epic application build, training-environment content, and clinical workflow design.
- Network, print-server, and Citrix/VDI back-end architecture (touched only where an endpoint symptom points to them).

Approach — the phase model

This playbook follows the same five-phase model used across the Healthcare Solutions category, so teams running multiple solutions see one consistent operating rhythm:

1. **Assess** — inventory the in-scope fleet, capture a readiness snapshot of current state, and identify every gap against the readiness standard. Nothing is assumed ready.
2. **Prepare / Remediate** — close the gaps: enroll missing devices, push the correct Epic client version, fix printing and peripherals, confirm Imprivata, and bring devices into compliance. All remediation is read-only / `-WhatIf` by default and only changes state when explicitly applied.
3. **Validate (go/no-go)** — run the validation sweep to produce a PASS/FAIL per device, roll the results into the readiness scorecard, and walk the go/no-go gate to a documented recommendation.
4. **Command Center (hypercare)** — stand up the command center with its roster, tiers, dispatch, and at-the-elbow model; run the triage runbook against live day-1 issues; execute the comms and escalation plan.
5. **Stabilize / Handover** — measure against the exit criteria, retire the command center, and hand the environment to steady-state service-desk ownership with documentation intact.

Named artifacts this playbook produces

Every artifact below is named with the convention `<Purpose> – <Scope> – <Stage>` and is referred to by that exact name throughout the playbook, so a name written in one document means the same object everywhere:

- **Endpoint Readiness Checklist – Clinical Fleet – Pre-Go-Live** — the device-level readiness standard.

- **Go/No-Go Gate – Endpoint Domain – Activation** — the aggregated decision checklist.
- **Readiness Scorecard – Clinical Fleet – Validation** — the per-device and rolled-up PASS/FAIL report (produced by `EPICGOLIVE-Report.ps1`).
- **Command Center Roster – Endpoint Support – Hypercare** — roles, tiers, dispatch, and on-call coverage.
- **Rapid-Fix Triage Runbook – Day-1 Endpoint – Hypercare** — the common-problem fix procedures.
- **Comms & Escalation Plan – Endpoint Domain – Activation** — who-tells-whom and the escalation ladder.
- **Hypercare Exit Criteria – Endpoint Domain – Handover** — the steady-state handover gate.
- **Readiness Device Group – Clinical Fleet – Validation** — the dynamic/assigned device group the sweep and remediation target (managed by `EPICGOLIVE-Set-ReadinessGroup.ps1`).

What you get

- **8 source documents** (this overview, requirements, category-wide global requirements, implementation, testing, best practices, troubleshooting, deliverables summary) plus an executive summary.
- **5 PowerShell scripts** that automate discovery, validation, reporting, remediation, and readiness-group management against Microsoft Intune / Microsoft Entra via Microsoft Graph.
- A **hero image** for the listing.

Related solutions (deep config lives here)

- **Epic client deployment & standardization solution** — for Hyperdrive/Hyperspace packaging, version management, and deployment rings.
- **Imprivata tap-and-go / SSO solution** — for OneSign agent, enrollment, and policy configuration.
- **Clinical workstation standardization solution** — for the master image, baseline, and hardware standards.

This playbook references those solutions wherever a step needs deep configuration; it focuses on getting the fleet **ready, validated, and supported** through the activation event itself.

2. Requirements — Epic Go-Live & Command Center Endpoint Readiness Playbook

This section defines the solution-specific requirements for getting the clinical endpoint fleet ready for an Epic go-live and running the command center through hypercare. **Category-wide prerequisites** (tenant, roles, licensing, HIPAA/HITRUST data handling and change control, maintenance windows, network, test device, rollback) are defined once in `global-requirements-healthcare-solutions.md` — read that document first; the requirements below assume it.

Each requirement follows the same four-part structure:

- **Description** — what the requirement is.

- **Why** — why it matters for an Epic activation.
- **Verify** — how to confirm it is already in place.
- **Implement if missing** — what to do when it is not.

Never assume any requirement is already satisfied — verify each one.

R1 — Authoritative in-scope device inventory

Description. A single authoritative list of every clinical endpoint that must be ready for the activation — clinical workstations, WOWs (workstations-on-wheels), exam-room PCs, shared/kiosk devices, and the clinical peripherals attached to them — scoped to the go-live's units, floors, departments, or facility.

Why. Readiness is meaningless without a denominator. The go/no-go gate reports "X of Y devices PASS"; if Y is wrong, the recommendation is wrong. Devices that exist on the floor but not in inventory are exactly the ones that fail on day 1 because nobody prepared them.

Verify. In **Microsoft Intune admin center > Devices > All devices**, filter to the activation's scope and reconcile the count against the Epic program's location/asset list and the network/DHCP view of what is actually on those floors. Run `EPICGOLIVE-Discovery.ps1` to produce a fleet snapshot and compare its device count to the expected count.

Implement if missing. Build the inventory by reconciling Intune/Entra device records, the asset-management/CMDB list, and a physical floor walk of the activation area. Tag in-scope devices so they can be grouped (see R10). Resolve every discrepancy (in Intune but not on the floor, on the floor but not in Intune) before validation begins.

R2 — Defined, measurable device readiness standard

Description. A written readiness standard — the **Endpoint Readiness Checklist – Clinical Fleet – Pre-Go-Live** — that states, for each device class, the exact conditions a device must meet to be "ready": required Epic client version, required peripherals, Imprivata tap functional, maximum acceptable login time, and required compliance/enrollment state.

Why. "Ready" must be objective and the same for everyone, or the validation sweep produces inconsistent results and the go/no-go gate cannot be defended. A measurable standard is also what makes the PowerShell validation possible — a script can only test a concrete threshold.

Verify. Confirm a current readiness checklist exists, is approved by the Epic program / Epic technical services (TS), and names concrete values (e.g., the exact target Epic client version string, the login-time target in seconds, the required compliance policy). Confirm each in-scope device class maps to a checklist profile.

Implement if missing. Author the checklist with the Epic program and Epic TS supplying the environment-specific values (client version, print pathways, login-time target). Use the artifact name above verbatim. Have it formally approved before it gates anything.

R3 — Correct Epic client version present on every device

Description. Every in-scope device runs the **exact** Epic client version (Hyperdrive/Hyperspace) the Epic program has designated for this go-live — no older build, no unexpected newer build.

Why. A version mismatch is one of the most common day-1 blockers: the client refuses to connect, prompts for an unwanted upgrade, or behaves differently than staff were trained on. Version drift across the fleet also makes triage unpredictable.

Verify. Run `EPICGOLIVE-Discovery.ps1` / `EPICGOLIVE-Validation.ps1` to read the installed Epic client version from each device and compare to the designated version from the readiness checklist (R2). In Intune, check **Apps > Monitor** for the deployment status of the Epic client app.

Implement if missing. Deploy or update the designated client version via the Epic client deployment solution (referenced — deep packaging/ring config lives there). Re-run discovery to confirm the version is correct fleet-wide. Do not hand-patch individual devices outside the managed deployment.

R4 — Functional clinical printing

Description. Every device can print to the printers its workflow needs — label/wristband printers, document printers, and any specialty printers (e.g., prescription, MAR) — through the print pathway the Epic environment uses.

Why. Printing failures (wristbands, labels, scripts) are a top day-1 ticket category and directly affect patient safety and throughput. A device that logs in fine but can't print a wristband is not ready.

Verify. Confirm printers are mapped/available on each device and that a test print succeeds through the intended pathway. Validate that print configuration is delivered consistently (e.g., via the print management method the environment uses) rather than manually per device.

Implement if missing. Map the required printers and confirm drivers/pathway are correct; coordinate with the print-server / Epic print team for the print-routing configuration. Document the print pathway in the readiness checklist so the triage runbook can reference it.

R5 — Attached peripherals present and working

Description. Every required clinical peripheral — barcode/label scanners, badge readers, signature pads, document scanners, card swipes — is physically attached, recognized by the OS, and functional on the device.

Why. Clinical workflows (medication administration, patient ID, document capture) depend on peripherals. A missing or non-functional scanner stops the workflow cold at the bedside.

Verify. Run `EPICGOLIVE-Validation.ps1` to check that expected peripherals enumerate on each device, and confirm a functional test (scan a barcode, tap a badge, sign on the pad) during the validation sweep. Cross-check the expected peripheral set per device class against the readiness checklist.

Implement if missing. Attach/replace missing or faulty peripherals, install/confirm drivers, and re-test. Where a peripheral is delivered by policy/driver package, confirm the package is applied. Record each device's expected peripheral set so validation knows what "complete" means.

R6 — Imprivata tap-and-go (SSO) functional

Description. Imprivata tap-and-go single sign-on works on every device that uses it: badge tap authenticates, fast user switching / proximity workflows behave as configured, and the Imprivata agent is healthy.

Why. Tap-and-go is how clinicians log in dozens of times a shift. If the tap fails, staff fall back to manual credentials, login times spike, and the floor slows to a crawl — the single most felt endpoint failure at go-live.

Verify. Confirm the Imprivata agent is installed and reporting healthy on each device, and perform a badge-tap test during the validation sweep. Check the readiness checklist for which device classes require tap-and-go.

Implement if missing. Install/repair the Imprivata agent and confirm enrollment/policy (deep OneSign config lives in the referenced Imprivata solution). Re-test the tap. Ensure a documented manual-login fallback exists for the triage runbook.

R7 — Login time within target

Description. The time from badge tap / credential entry to a usable Epic session is at or below the maximum defined in the readiness checklist (R2) for each device class.

Why. Slow logins are a leading source of clinician frustration and lost productivity at go-live and a frequent disguised symptom of deeper problems (profile bloat, GPO/login-script delays, resource contention). A measured target turns "it feels slow" into a pass/fail.

Verify. Measure login time on representative devices per class during validation; capture the metric in the readiness scorecard. Compare to the checklist target.

Implement if missing. Investigate and remediate the dominant contributor (roaming profile size, startup apps, login scripts, GPO processing, antivirus on-access scope, disk/RAM on the device). Re-measure until within target. Persistent outliers are flagged in the scorecard and weighed at the go/no-go gate.

R8 — Devices enrolled and compliant

Description. Every in-scope device is enrolled in Microsoft Intune, is the expected managed identity in Microsoft Entra, and is **compliant** with the assigned compliance policy (encryption, OS build, security baseline, etc.).

Why. A not-enrolled or non-compliant device may be blocked by Conditional Access from reaching Epic or its dependencies, may lack the management channel needed to push fixes, and is invisible to the readiness tooling. "Not enrolled" is a classic day-1 surprise.

Verify. Run `EPICGOLIVE-Discovery.ps1` to list enrollment and compliance state per device from Intune/Entra via Graph. In Intune, review **Devices > Compliance** and **Devices > Conditional access** impact for the in-scope group.

Implement if missing. Enroll missing devices and remediate compliance failures (the cause is reported per device). Use `EPICGOLIVE-Remediation.ps1` (read-only / `-WhatIf` by default; `-Apply` to act) for the safe, supported remediation actions. Re-run discovery until the in-scope group is fully enrolled and compliant.

R9 — Command-center operating model defined

Description. A defined command-center structure before go-live: the **Command Center Roster – Endpoint Support – Hypercare** (roles, support tiers, dispatch model, at-the-elbow assignments, on-call coverage), the **Rapid-Fix Triage Runbook – Day-1 Endpoint – Hypercare**, and the **Comms & Escalation Plan – Endpoint Domain – Activation**.

Why. A command center improvised on day 1 cannot keep up with a live clinical floor. Defined roles, a dispatch model, and a triage runbook are what let the team resolve high-volume issues in minutes and escalate the rest cleanly.

Verify. Confirm the roster, triage runbook, and comms/escalation plan exist, name real people/roles for the activation window, and have been walked through with the command-center and at-the-elbow staff. Confirm a ticketing/dispatch channel is ready.

Implement if missing. Build the three artifacts (templates and structure are in `03-implementation.md`), staff them for the activation window including off-hours, and dry-run the dispatch and escalation paths before go-live.

R10 — Readiness device group for targeting

Description. A managed device group — the **Readiness Device Group – Clinical Fleet – Validation** — in Intune/Entra that contains exactly the in-scope devices, so the validation sweep, remediation, and any deployments target the right set and nothing else.

Why. Targeting by group keeps validation, remediation, and reporting scoped to the activation. A wrong or stale group either misses devices (false readiness) or sweeps unrelated production devices (noise and risk).

Verify. Confirm the group exists in **Microsoft Entra admin center > Groups** (or **Intune > Groups**), that its membership matches the R1 inventory, and that its name matches the artifact name verbatim. Run `EPICGOLIVE-Set-ReadinessGroup.ps1 -WhatIf` to preview the intended membership against current.

Implement if missing. Create the group and populate it from the R1 inventory (dynamic membership rule on the in-scope tag/attribute, or assigned membership for a fixed list). Use `EPICGOLIVE-Set-ReadinessGroup.ps1` (preview with `-WhatIf` first) to manage membership. Keep it in sync as devices are added/removed during prep.

R11 — Go/no-go gate and decision authority

Description. A defined **Go/No-Go Gate – Endpoint Domain – Activation** checklist and a named decision authority (typically the Epic program lead, advised by the endpoint lead) who accepts the endpoint domain's go/no-go recommendation.

Why. Validation produces data; someone must turn data into a decision and own it. Without a defined gate and authority, readiness debates happen ad hoc at the worst time, and conditional-go conditions go untracked.

Verify. Confirm the gate checklist exists, defines the PASS thresholds (e.g., minimum percent of in-scope devices at PASS, zero unresolved safety-critical failures), and names the decision authority and the meeting/time at which the gate is walked.

Implement if missing. Author the gate with explicit thresholds tied to the readiness scorecard, identify the decision authority, and schedule the go/no-go review inside the activation timeline. Define how conditional-go items are tracked into hypercare.

R12 — Hypercare exit criteria defined

Description. A defined **Hypercare Exit Criteria – Endpoint Domain – Handover** — the measurable conditions under which the command center retires and ownership moves to steady-state service desk (e.g., endpoint ticket volume below a threshold for N consecutive days, no open safety-critical endpoint issues, knowledge transferred).

Why. Without exit criteria, hypercare either ends arbitrarily (leaving the service desk unprepared) or never ends (burning out the command-center team). Clear criteria make the handover a decision, not a default.

Verify. Confirm exit criteria exist, are measurable, and are agreed with service-desk leadership who will inherit ownership.

Implement if missing. Define the criteria with command-center and service-desk leadership, tie them to the same metrics the command center already tracks, and document the handover package contents (open items, known issues, updated runbook). See `03-implementation.md` Phase 5.

3. Implementation — Epic Go-Live & Command Center Endpoint Readiness Playbook

This is the operational walkthrough. It follows the five-phase model — **Assess** → **Prepare/Remediate** → **Validate (go/no-go)** → **Command Center (hypercare)** → **Stabilize/Handover** — and produces the named artifacts defined in `01-solution-overview.md`. Every step that uses a console gives the full navigation path. Confirm the prerequisites in `02-requirements.md` and `global-requirements-healthcare-solutions.md` before starting; never assume a prerequisite is satisfied.

> ***Safety posture.*** *All read-only steps (discovery, validation, reporting) can run any time. Every state-changing step happens only inside an approved maintenance window (GR4), against the readiness device group (R10), with a documented rollback (GR7), and after a successful test-device run (GR6). Remediation tooling defaults to `-WhatIf`; it changes nothing until you pass `-Apply`.*

Phase 1 — Assess

Goal: know exactly what is in scope and where every device stands against the readiness standard. Assume nothing is ready.

Step 1.1 — Confirm scope and build the in-scope inventory (R1)

1. Get the activation's scope from the Epic program: which units, floors, departments, or facility, and the activation date/time.

2. In **Microsoft Intune admin center > Devices > All devices**, filter to the scope (by group, OU, naming, or tag) and export the list.

3. Reconcile that list against the asset/CMDB list and a physical floor walk of the activation area. Resolve every discrepancy (in Intune but not on the floor; on the floor but not in Intune).

4. Record the agreed denominator — the count the go/no-go gate will report against.

Step 1.2 — Capture the readiness snapshot

1. Connect to Microsoft Graph and run `EPICGOLIVE-Discovery.ps1` scoped to the inventory. It produces a fleet readiness snapshot: per device, the enrollment state, compliance state, OS build, detected Epic client version, Imprivata agent presence, and last check-in — all read-only.

2. Save the snapshot as the **before** baseline (it also serves the GR7 rollback baseline).

Step 1.3 — Define / confirm the readiness standard (R2)

1. Confirm the **Endpoint Readiness Checklist – Clinical Fleet – Pre-Go-Live** exists and carries the environment-specific values supplied by the Epic program / Epic TS:

• exact target **Epic client version** string;

• required **printers / print pathway** per device class;

• required **peripherals** per device class;

• **Imprivata tap-and-go** required (yes/no) per device class;

• maximum acceptable **login time** (seconds) per device class;

• required **compliance policy** / enrollment state.

1. Map each device class in the inventory to a checklist profile.

Step 1.4 — Gap analysis

1. Compare the snapshot (1.2) to the standard (1.3). Produce the gap list: not-enrolled, non-compliant, wrong Epic version, missing Imprivata agent, etc.

2. This gap list drives Phase 2. Track it in the same place the go/no-go gate will read from.

Step 1.5 — Create the readiness device group (R10)

1. In **Microsoft Entra admin center > Groups > New group** (or **Intune > Groups**), create **Readiness Device Group – Clinical Fleet – Validation** (name verbatim).

2. Populate it from the inventory — a dynamic membership rule on the in-scope tag/attribute, or assigned membership for a fixed list.

3. Preview and manage membership with `EPICGOLIVE-Set-ReadinessGroup.ps1 -WhatIf`; confirm the preview matches the inventory before applying.

Phase 2 — Prepare / Remediate

Goal: close every gap from Phase 1 so the fleet can pass validation. All actions are scheduled into maintenance windows, tested first, and reversible.

Step 2.1 — Remediate enrollment and compliance (R8)

1. Review compliance failures in **Microsoft Intune admin center > Devices > Compliance** for the readiness group, and the Conditional Access impact under **Devices > Conditional access**.

2. Run `EPICGOLIVE-Remediation.ps1` (default `-WhatIf`) to preview the safe, supported remediation actions per device (e.g., re-sync, re-evaluate compliance, trigger enrollment follow-up). Review the plan.

3. On the test device (GR6), run with `-Apply`; confirm the device becomes enrolled/compliant.

4. In an approved window, run with `-Apply` against the readiness group. Re-run `EPICGOLIVE-Discovery.ps1` to confirm.

Step 2.2 — Deploy the correct Epic client version (R3)

1. Using the **Epic client deployment solution** (referenced — deep packaging/ring config lives there), deploy the designated client version to the readiness group.

2. Track deployment under **Microsoft Intune admin center > Apps > Monitor > App install status**.

3. Re-run `EPICGOLIVE-Discovery.ps1`; confirm every device reports the exact designated version.

Step 2.3 — Fix printing (R4)

1. Confirm each device has the required printers/pathway from the checklist. Coordinate the print-routing configuration with the print-server / Epic print team.

2. Perform a test print per device class through the intended pathway (label/wristband, document, specialty).

3. Record the print pathway in the readiness checklist so the triage runbook (Step 4.3) can reference it.

Step 2.4 — Fix peripherals (R5)

1. For each device class, confirm the expected peripheral set enumerates and functions (scan a barcode, tap a badge, sign a pad).

2. Attach/replace missing or faulty peripherals; install/confirm drivers or the driver package.

3. Re-test until each device's expected peripheral set is complete.

Step 2.5 — Confirm Imprivata tap-and-go (R6)

1. Confirm the Imprivata agent is installed and healthy on each device that requires tap-and-go (deep OneSign config lives in the referenced Imprivata solution).

2. Perform a badge-tap test; confirm fast user switching / proximity behavior matches policy.

3. Confirm the documented manual-login fallback works (the triage runbook depends on it).

Step 2.6 — Address login-time outliers (R7)

1. Measure login time on representative devices per class. For any device/class over target, investigate the dominant contributor (profile size, startup apps, login scripts, GPO processing, antivirus scope, device RAM/disk).

2. Remediate and re-measure until within target. Flag persistent outliers for the go/no-go gate.

Phase 3 — Validate (go/no-go)

Goal: prove readiness with evidence and convert it to a documented go/no-go recommendation.

Step 3.1 — Run the validation sweep

1. Run `EPICGOLIVE-Validation.ps1` against the readiness group. For each device it returns a **PASS/FAIL** against the readiness checklist criteria: Epic version correct, enrolled+compliant, Imprivata present, peripherals enumerated, plus the data points feeding printing and login-time checks.

2. Re-run as needed after late fixes; the sweep is read-only and idempotent.

Step 3.2 — Produce the readiness scorecard

1. Run `EPICGOLIVE-Report.ps1` to roll the validation results into the **Readiness Scorecard – Clinical Fleet – Validation** and export to CSV: per-device PASS/FAIL with reasons, and rolled-up percentages by device class, unit, and overall.

2. Confirm the export contains only device/posture metadata (no ePHI — GR3).

Step 3.3 — Walk the go/no-go gate (R11)

1. Convene the go/no-go review at the scheduled time with the named decision authority.

2. Walk the **Go/No-Go Gate – Endpoint Domain – Activation** against the scorecard:

• Is the percent of in-scope devices at PASS at or above the defined threshold?

• Are there **zero unresolved safety-critical failures** (e.g., wristband printing on the affected unit, tap-and-go for med admin)?

• Are remaining FAILs documented with owner, fix plan, and risk?

1. Record the recommendation: **Go**, **Conditional-Go** (with tracked conditions carried into hypercare), or **No-Go** (with the gap that must close). Capture who decided and when.

Phase 4 — Command Center (hypercare)

Goal: run the activation floor — triage at clinical speed, escalate cleanly, communicate clearly.

Step 4.1 — Stand up the command center (R9)

1. Activate the **Command Center Roster – Endpoint Support – Hypercare**:

• **Command-center lead** — owns the endpoint domain on the floor; the single point for the Epic program.

• **Dispatcher(s)** — receive issues (ticket queue, phone, radio/chat), classify, and assign to the right tier/walker; track time-to-resolve.

• **Tier-1 at-the-elbow (ATE) / floor walkers** — physically on the units; run the triage runbook for common issues.

• **Tier-2 endpoint engineers** — handle issues the runbook escalates (imaging, complex peripheral/print, deeper SSO).

• **Tier-3 / vendor & application liaisons** — bridge to Epic application, Imprivata, network, and print teams.

1. Confirm support **tiers** and the **dispatch model**: how an issue flows from report → dispatcher → ATE → Tier-2 → Tier-3, with target response times per tier.

2. Set the at-the-elbow coverage map: which walker covers which unit/shift, including nights and weekends.

Step 4.2 — Activate comms and escalation (R9)

1. Stand up the **Comms & Escalation Plan – Endpoint Domain – Activation**:

• The dispatch channel (ticket queue + real-time chat/radio bridge).

• A scheduled **status cadence** (e.g., huddles at shift change) reporting endpoint ticket volume, top issues, and trends to the Epic program.

• The **escalation ladder**: ATE → command-center lead → Epic program; and the cross-team path to application/Imprivata/network/print Tier-3.

• Clear criteria for what is **safety-critical** and gets immediate top-of-ladder escalation.

Step 4.3 — Run the rapid-fix triage runbook

Use the **Rapid-Fix Triage Runbook – Day-1 Endpoint – Hypercare** for the high-volume problems. Each entry below is a quick-fix path; deep diagnostics are in `06-troubleshooting.md`.

A. Login / SSO failure (tap doesn't log in)

1. Confirm the badge reader is connected and lit; re-tap.

2. Confirm the Imprivata agent is running (tray/health); restart it if hung.

3. Fall back to **manual credential login** so the clinician can work *now*; log the device for Tier-2.

4. If many devices on one unit fail at once, escalate as a possible Imprivata/network event (don't fix device-by-device).

B. Printing failure (wristband/label/document won't print)

1. Confirm the printer is powered, online, and has media; clear any error.

2. Confirm the device has the correct printer mapped (per the checklist print pathway); re-add if missing.

3. Print a test page through the intended pathway; if it fails server-side, escalate to print/Epic print Tier-3.

4. For wristband/safety printing, treat as **safety-critical** — escalate immediately if not resolved in minutes.

C. Device not enrolled / non-compliant (can't reach Epic)

1. Confirm the device is the expected managed device and is checked-in (have Tier-2 verify in Intune).

2. Trigger a sync / re-evaluate compliance; for a quick fix, run `EPICGOLIVE-Remediation.ps1 -Apply` against that device.

3. If it cannot be brought compliant quickly, **swap in a known-good ready device** so care continues; remediate the failed device off the floor.

D. Slow performance / slow login

1. Check for an obvious local cause (low disk, runaway process, pending reboot); reboot if appropriate.

2. Compare against peers on the same unit — if widespread, escalate as a back-end/resource event, not a per-device fix.

3. For a single slow device, swap in a ready spare and send the slow device to Tier-2.

Step 4.4 — Keep the picture current

1. The dispatcher maintains live counts by problem type; re-run `EPICGOLIVE-Report.ps1` periodically to track the fleet's readiness/posture during hypercare.

2. Feed recurring/new issues back into the triage runbook so the next shift inherits the fix.

Phase 5 — Stabilize / Handover

Goal: retire the command center on defined criteria and hand a documented environment to steady-state ownership.

Step 5.1 — Measure against exit criteria (R12)

1. Track the **Hypercare Exit Criteria – Endpoint Domain – Handover** — e.g., endpoint ticket volume below the threshold for N consecutive days, no open safety-critical endpoint issues, all conditional-go conditions closed, knowledge transferred.

2. Review the criteria with service-desk leadership who will inherit ownership.

Step 5.2 — Execute the handover

1. Assemble the handover package: open items with owners, known issues and workarounds, the updated triage runbook, the final readiness scorecard, and the as-built notes (final Epic client version, print pathway, Imprivata state).

2. Walk the service desk through the package and the top recurring issues; confirm they can run the relevant triage steps.

3. Transfer ownership of the readiness device group and the standing tooling/reports.

Step 5.3 — Retire and capture lessons

1. Formally stand down the command center on the decision authority's sign-off.

2. Run a brief retrospective; capture what to change for the next wave/site/upgrade. The artifacts (checklist, gate, roster, runbook, scorecard) are reusable — update them with what you learned so the next activation starts ahead.

5. Testing & Validation — Epic Go-Live & Command Center Endpoint Readiness Playbook

Testing in this playbook has two layers. **Solution validation** proves the fleet meets the readiness standard and feeds the go/no-go gate (the validation sweep). **Operational rehearsal** proves the command center, triage runbook, and escalation paths work *before* the floor goes live. Both are covered here. All tests are read-only unless explicitly a remediation rehearsal, which runs on the test device first (GR6).

1. The validation sweep (per-device PASS/FAIL)

The validation sweep is the primary test that proves device readiness. Run `EPICGOLIVE-Validation.ps1` against the **Readiness Device Group – Clinical Fleet – Validation**; it returns a PASS/FAIL per device against the **Endpoint Readiness Checklist – Clinical Fleet – Pre-Go-Live**.

For each device, confirm the sweep evaluates:

#	Check	PASS condition
1	Enrollment	Device enrolled in Intune, expected managed identity in Entra
2	Compliance	Compliant with the assigned compliance policy
3	Epic client version	Installed version equals the designated version (exact string)
4	Imprivata agent	Agent present and reporting healthy (for classes requiring tap)
5	Peripherals	Expected peripheral set for the device class enumerates
6	Printing	Required printer(s)/pathway present (functional test confirmed in sweep)
7	Login time	Measured login time at or below the class target
8	Check-in	Device has checked in recently (not stale/offline)

How to run and confirm:

1. Connect to Microsoft Graph with the read-only scopes.
2. Run `EPICGOLIVE-Validation.ps1` scoped to the readiness group.
3. Confirm every device returns a clear PASS or FAIL with a reason for each FAIL.
4. Re-run after any late fix — the sweep is idempotent and read-only.

2. Readiness scorecard (aggregate evidence)

Run `EPICGOLIVE-Report.ps1` to roll the sweep results into the **Readiness Scorecard – Clinical Fleet – Validation** (CSV).

Confirm the scorecard shows:

- Per-device PASS/FAIL with the specific failing check(s).
- Roll-ups: percent PASS by device class, by unit/department, and overall.
- A count of unresolved **safety-critical** failures (wristband/label printing on affected units, tap-and-go for medication administration).
- Only device/posture metadata — **no ePHI** (GR3). Verify this before sharing the export.

The scorecard is the evidence the go/no-go gate reads. Confirm its numbers reconcile to the in-scope denominator from R1 (the count of devices reported equals the agreed inventory count).

3. Functional spot-tests (human-in-the-loop)

Some checks need a person at the device. During validation, walk a representative sample per device class and confirm hands-on:

1. **Tap-and-go** — badge tap logs the test user into a usable Epic session within the login-time target; fast user switching behaves per policy.
2. **Manual-login fallback** — credential login works when tap is bypassed (the triage runbook depends on this).
3. **Wristband/label print** — a test wristband/label prints correctly through the intended pathway.
4. **Document print** — a test document prints to the mapped document printer.
5. **Scanner** — a barcode scan registers; **signature pad** captures; **document scanner** scans (per the device class's expected peripherals).

Record results against the readiness checklist. Any safety-critical functional failure (wristband print, med-admin tap) is a go/no-go blocker until resolved.

4. Test-device rehearsal of remediation (safe-change proof)

Before any `-Apply` remediation touches the fleet, prove it on the test device (GR6):

1. Run `EPICGOLIVE-Remediation.ps1` with default `-WhatIf` against the test device; review the planned actions.
2. Run with `-Apply` against the test device only.
3. Confirm the device reaches the intended state (re-run `EPICGOLIVE-Discovery.ps1`).
4. Confirm the rollback (GR7) reverses the change on the test device.
5. Only then expand to the readiness group inside an approved window (GR4).

5. Command-center operational rehearsal (dry run)

Test the operating model before go-live, not during it.

1. **Roster walkthrough** — confirm the **Command Center Roster – Endpoint Support – Hypercare** names real people for every role and shift, including nights/weekends, and that each person knows their tier and coverage.
2. **Dispatch dry run** — inject sample issues (one per triage category: login/SSO, printing, not-enrolled, slow) and confirm each flows report → dispatcher → correct tier within the target response time.
3. **Triage runbook walkthrough** — have ATE staff execute each runbook entry against the test device(s) and confirm the steps are accurate and fast.
4. **Escalation drill** — escalate a simulated safety-critical issue up the ladder (ATE → command-center lead → Epic program; and to the right Tier-3 vendor/app team) and confirm it reaches the right person on the first hop.
5. **Comms cadence** — run one practice status huddle using the real reporting format and the live counts from `EPICGOLIVE-Report.ps1`.

Capture any gap (a role unstaffed for nights, a dispatch path that misroutes, a runbook step that's wrong) and fix it before the activation.

6. Go/no-go gate validation

Confirm the **Go/No-Go Gate – Endpoint Domain – Activation** is ready to be walked:

1. Thresholds are explicit (minimum percent PASS; zero unresolved safety-critical failures).
2. The decision authority is named and available at the scheduled review.
3. The conditional-go tracking mechanism exists (conditions carry into hypercare with owners).
4. The gate reads from the readiness scorecard, and the scorecard is current.

7. Exit-criteria validation (hypercare → steady state)

Confirm the **Hypercare Exit Criteria – Endpoint Domain – Handover** are measurable and instrumented:

1. The metrics they depend on (endpoint ticket volume, open safety-critical count) are actually being tracked during hypercare.
2. Service-desk leadership has agreed to the criteria and the handover package contents.
3. There is a defined "measure, then decide" review rather than an arbitrary end date.

Validation checklist (summary)

- Validation sweep returns PASS/FAIL for 100% of in-scope devices, reasons on every FAIL.
- Scorecard reconciles to the R1 inventory count; contains no ePHI.

- Safety-critical functional tests (wristband print, med-admin tap) pass on every affected unit.
- Remediation rehearsed and rolled back on the test device before any fleet `-Apply`.
- Command-center roster fully staffed including off-hours; dispatch dry run passed.
- Triage runbook walked through and accurate; escalation drill reached the right people.
- Go/no-go gate thresholds explicit, authority named, scorecard current.
- Hypercare exit criteria measurable and instrumented; service desk agreed.

6. Troubleshooting — Epic Go-Live & Command Center Endpoint Readiness Playbook

This section is the deep-diagnostic companion to the rapid-fix triage runbook in `03-implementation.md` (Step 4.3). The runbook gets the clinician working *now*; this section is how Tier-2/Tier-3 root-cause and permanently resolve the failure. Each entry follows **Symptom** → **Likely causes** → **Diagnose** → **Resolve** → **Prevent**. Deep configuration for the Epic client, Imprivata, and the workstation image lives in the referenced solutions; this focuses on endpoint-side diagnosis during an activation.

> ***Pattern first.*** *Before deep-diving a single device, check whether many devices on one unit are failing the same way at the same time. A cluster is almost always a back-end event (Imprivata appliance, network/VLAN, print server, Conditional Access change) — fix the cause once instead of N devices. The dispatcher's live counts are the fastest way to see a pattern.*

A. Login / SSO — badge tap doesn't log in

Symptom. Clinician taps badge, nothing happens, wrong/delayed prompt, or session won't open.

Likely causes. Badge reader disconnected/failed; Imprivata agent stopped or unhealthy; agent can't reach the Imprivata appliance (network); badge not enrolled/mapped; Conditional Access or compliance blocking the sign-in; time skew on the device.

Diagnose.

1. Confirm the badge reader is connected and recognized by the OS; test with a known-good badge.
2. Check the Imprivata agent health/tray; confirm the service is running.
3. Test reachability from the device to the Imprivata appliance and to the Microsoft sign-in endpoints (GR5 dependency map).
4. In **Microsoft Entra admin center > Monitoring > Sign-in logs**, find the user/device sign-in and read the failure reason (e.g., Conditional Access, compliance).
5. Check device time sync (Kerberos/auth fails on skew).

Resolve.

- Reader fault → reseat/replace reader; confirm driver.
- Agent hung → restart the Imprivata agent/service; if persistent, repair the agent (referenced Imprivata solution).

- Reachability → fix the network path to the appliance/endpoints with the network team.
- Conditional Access/compliance block → resolve compliance via `EPICGOLIVE-Remediation.ps1 -Apply` for that device; confirm the device is in the readiness group and compliant.
- Time skew → correct time source.

Prevent. Validate Imprivata agent health and a live badge tap in the pre-go-live sweep (R6); confirm the manual-login fallback works so a single tap failure never stops care.

B. Printing — wristband / label / document won't print

Symptom. Print job fails, prints to the wrong device, or nothing happens. Wristband/label failures are **safety-critical**.

Likely causes. Printer offline/out of media; printer not mapped on the device; wrong/corrupt driver; print pathway / print-server problem (server-side); wrong default for the workflow.

Diagnose.

1. Confirm the physical printer is powered, online, loaded, and error-free.
2. Confirm the device has the correct printer mapped per the readiness checklist print pathway.
3. Print a test page locally; if it fails, suspect driver/mapping. If it succeeds locally but the Epic job fails, suspect the print pathway/server (escalate to print/Epic print Tier-3).
4. Compare against a peer device on the same unit — if all fail, it's the pathway/server, not the device.

Resolve.

- Hardware → clear error, load media, bring online.
- Mapping/driver → re-add the printer / reinstall the correct driver via the managed method.
- Pathway/server → escalate to print/Epic print team Tier-3; do not hand-patch around a server fault.
- For safety-critical wristband/label printing, escalate immediately if not resolved in minutes; route the affected workflow to a known-good printer in the interim.

Prevent. Confirm printing per device class in Phase 2 (R4) with a functional test print, and document the print pathway in the checklist so triage isn't guessing.

C. Enrollment / compliance — device can't reach Epic or its dependencies

Symptom. Device is blocked from Epic/dependencies, shows non-compliant, or isn't manageable.

Likely causes. Device never enrolled / dropped enrollment; failing a compliance setting (encryption, OS build, baseline); Conditional Access blocking non-compliant devices; stale/no check-in; wrong/duplicate device identity in Entra.

Diagnose.

1. Run `EPICGOLIVE-Discovery.ps1` for the device; read enrollment, compliance, and last check-in.

2. In **Microsoft Intune admin center > Devices > [device] > Compliance**, read the specific failing setting.

3. Check for duplicate/stale Entra device records.

4. Confirm the device is in the **Readiness Device Group – Clinical Fleet – Validation** (untargeted devices won't get policy/fixes).

Resolve.

• Specific compliance failure → remediate the named setting; run `EPICGOLIVE-Remediation.ps1 -Apply` for the safe, supported actions (re-sync, re-evaluate).

• Not enrolled → trigger/repair enrollment; re-run discovery.

• Stale/duplicate identity → clean up the stale record (change-controlled).

• Can't fix fast on the floor → **swap in a known-good ready device** so care continues; remediate the failed one off the floor.

Prevent. Drive enrollment+compliance to 100% in Phase 2 (R8) and verify in the validation sweep; keep the readiness group synced (R10).

D. Performance — slow login or slow session

Symptom. Login or Epic session is slow enough to impede care.

Likely causes (device-local). Low disk/RAM, pending reboot, runaway process, oversized roaming profile, heavy startup apps, antivirus on-access scope, slow GPO/login-script processing. **(Back-end).** Resource contention on shared infrastructure, network latency, presentation-layer load.

Diagnose.

1. Compare the device to peers on the same unit. **Widespread = back-end**; **isolated = device-local.**

2. Device-local: check disk/RAM, pending reboots, startup/login-script time, profile size, AV scope.

3. Back-end: escalate to the infrastructure/Epic team with the pattern (which units, what time, how many devices).

Resolve.

• Device-local → reboot if appropriate; trim startup/profile; correct AV scope; if a single device stays slow, swap in a ready spare and send it to Tier-2.

• Back-end → escalate the pattern; do not chase it device-by-device.

Prevent. Set and measure a login-time target per class (R7); remediate outliers in Phase 2 before go-live.

E. Wrong / missing Epic client version

Symptom. Client refuses to connect, prompts for an unexpected upgrade, or behaves differently than training.

Likely causes. Deployment didn't reach the device (not in readiness group, failed install, no check-in); version drift from a prior build; an unmanaged manual install.

Diagnose.

1. Run `EPICGOLIVE-Discovery.ps1` / `EPICGOLIVE-Validation.ps1`; compare the detected version to the designated version (exact string).
2. In **Microsoft Intune admin center > Apps > Monitor > App install status**, check the Epic client deployment result for the device.
3. Confirm the device is in the readiness group.

Resolve.

- Redeploy/repair the designated version via the Epic client deployment solution (referenced); re-run discovery to confirm.
- Remove unmanaged manual installs; let the managed deployment own the version.

Prevent. Verify the exact version fleet-wide in Phase 2 (R3) and re-confirm in the sweep before the gate.

F. Peripheral not working — scanner / badge reader / signature pad

Symptom. A required peripheral doesn't enumerate or doesn't function.

Likely causes. Disconnected/failed hardware; missing/wrong driver; driver package not applied; USB/port issue; device not in the group that delivers the driver package.

Diagnose.

1. Run `EPICGOLIVE-Validation.ps1` to see whether the expected peripheral enumerates.
2. Physically confirm connection; try another port; test the peripheral on a known-good device to isolate hardware vs. device.
3. Confirm the driver/driver package is present and applied.

Resolve.

- Hardware → reseat/replace.
- Driver → install/repair via the managed package; confirm the device is targeted.
- Re-test the function (scan/tap/sign) before declaring resolved.

Prevent. Define the expected peripheral set per class and validate it in Phase 2 (R5).

G. Tooling / script issues

Symptom. A solution script errors or returns incomplete data.

Likely causes. Microsoft Graph not connected or missing scopes; insufficient admin role; throttling on large fleets; readiness group empty/stale; ePHI-minimization filter mis-scoped.

Diagnose.

1. Confirm Graph connection and that the required scopes are consented (read-only for discovery/validation/report; minimal write for remediation/group).
2. Confirm your role is sufficient and least-privilege (GR1).

3. For large fleets, check for Graph throttling; re-run with paging/retry.

4. Confirm the readiness group is populated (`EPICGOLIVE-Set-ReadinessGroup.ps1 -WhatIf`).

Resolve.

• Reconnect/consent the correct scopes; assign the needed role; let the script's retry handle throttling; repopulate the group.

• If a report looks empty, confirm the scope/group, not the script, is the cause.

Prevent. Establish Graph connectivity, scopes, and roles as part of GR1 before Phase 1; rehearse the scripts on the test device (GR6).

H. Command-center / process issues

Symptom. Issues pile up, misroute, or escalate slowly during hypercare.

Likely causes. A shift/unit unstaffed; engineers swarming the loud issue instead of being dispatched; unclear safety-critical criteria; cross-team Tier-3 contacts not pre-established; runbook steps stale.

Diagnose & resolve.

1. Check the roster coverage map against the live floor — fill any unstaffed shift/unit immediately.

2. Reinforce the dispatch model: dispatcher classifies and assigns; engineers don't self-cluster.

3. Re-confirm what counts as safety-critical and that it jumps the ladder.

4. Re-confirm Tier-3 contacts (Epic app, Imprivata, network, print) are reachable on the first hop.

5. Update the triage runbook with any newly found fix so the next shift inherits it.

Prevent. Run the command-center dry run (`05-testing.md` §5) — roster walkthrough, dispatch dry run, triage walkthrough, escalation drill, comms cadence — *before* go-live, and fix every gap then.

7. Best Practices — Epic Go-Live & Command Center Endpoint Readiness Playbook

These practices are the accumulated discipline that separates a smooth endpoint activation from a chaotic one. They reinforce the phase model and the named artifacts; apply them across Assess → Prepare/Remediate → Validate → Command Center → Stabilize.

Readiness standard and validation

• **Make "ready" measurable, then automate the check.** Every readiness criterion should be a concrete threshold (exact client version string, login-time in seconds, named compliance policy) so a script can test it and two people get the same answer. Subjective readiness is unprovable readiness.

• **Validate against a denominator you trust.** Reconcile the in-scope inventory three ways (Intune/Entra, asset/CMDB, physical floor walk) before validation. A wrong denominator makes a 98%-ready report a lie.

• **Re-run the sweep, don't patch the report.** When a device is fixed, re-run `EPICGOLIVE-Validation.ps1`; never hand-edit a PASS into the scorecard. The scorecard's credibility at the go/no-go gate depends on it being machine-generated.

• **Separate read-only from state-changing, always.** Discovery, validation, and reporting are safe any time. Remediation defaults to `-WhatIf` and only acts with `-Apply`, in a window, after a test-device run. Keep that line bright.

• **Weight safety-critical failures differently.** A 99% PASS rate with one broken wristband printer on the affected unit is not a go. Track safety-critical checks (wristband/label print, med-admin tap) as a separate gate item, not blended into the percentage.

Targeting and change safety

• **Target everything through the readiness device group.** Validation, remediation, and deployments all scope to **Readiness Device Group – Clinical Fleet – Validation**. Never run an ad-hoc scope at go-live — group targeting is what stops a fix from sweeping unrelated production devices.

• **Keep the group in sync with reality.** Devices get added and swapped during prep; re-preview membership with `EPICGOLIVE-Set-ReadinessGroup.ps1 -WhatIf` and reconcile to the inventory regularly.

• **Snapshot before you change.** The discovery/report scripts produce the before-state that is also your rollback baseline (GR7). Capture it before any `-Apply`.

• **Respect the freeze.** Around go-live there is usually a change freeze. Default to read-only, schedule any change into an approved window, and clear it with the Epic program.

Command center

• **Staff for the floor, including nights and weekends.** The most common command-center failure is an unstaffed night shift on day 1. The roster covers every unit and every shift or it isn't done.

• **Put bodies on the units (at-the-elbow).** A nurse who can turn and ask a walker resolves an issue in seconds that would otherwise become a ticket. ATE coverage is the highest-leverage staffing decision.

• **Dispatch, don't swarm.** One dispatcher classifies and routes; engineers don't self-assign or cluster on the loud issue. A clear dispatch model keeps response times predictable.

• **Fix now, repair later.** At the bedside, restore the clinician's ability to work *immediately* — manual-login fallback, swap in a ready spare — then remediate the failed device off the floor. Don't make a clinician wait while you root-cause.

• **Watch for patterns, not just tickets.** Many identical failures on one unit at one time is a back-end event (Imprivata, network, print server), not N device problems. Escalate the pattern; don't fix it device-by-device.

• **Feed the runbook forward.** When a new fix is found, write it into the **Rapid-Fix Triage Runbook – Day-1 Endpoint – Hypercare** so the next shift inherits it. The runbook should be smarter at the end of hypercare than at the start.

Comms and escalation

• **Define safety-critical up front.** Everyone — dispatcher, ATE, lead — must know which issues jump straight to the top of the ladder (anything blocking medication administration, patient ID, or wristband printing). Ambiguity here costs minutes that matter.

• **One voice to the program.** The command-center lead is the single endpoint-domain contact for the Epic program. Many channels into the program create conflicting status.

• **Report trends on a cadence.** A short, scheduled huddle (shift change) reporting volume, top issues, and direction beats a flood of one-off updates and gives leadership a real picture.

• **Escalate cleanly across teams.** Pre-establish the Tier-3 contacts (Epic application, Imprivata, network, print) so a cross-team escalation reaches the right person on the first hop, not after three transfers.

Compliance and governance

• **No ePHI in any export.** Reports carry device and posture metadata only. Verify every CSV before it leaves the team (GR3).

• **Change-record every state change.** Even at go-live speed, remediations and deployments go through change control with a recorded rollback. Speed and auditability are not in tension if the runbook is prepared.

• **Least privilege, even under pressure.** Use the minimum role/scope needed; don't reach for Global Admin to move faster. Keep a break-glass account for genuine emergencies.

Handover and reuse

• **Exit on criteria, not on exhaustion.** Retire the command center when the **Hypercare Exit Criteria – Endpoint Domain – Handover** are met — not when the team is tired and not "two weeks, because that's what we said." Measure, then decide.

• **Hand over a package, not a hallway conversation.** Open items with owners, known issues with workarounds, the updated runbook, the final scorecard. The service desk should be able to run day-N without the command-center team in the room.

• **Treat every artifact as reusable.** The checklist, gate, roster, runbook, and scorecard are assets for the next wave, site, or upgrade. Update them with what you learned so the next activation starts ahead of where this one did.

• **Run the retrospective while it's fresh.** Capture what to change immediately after stand-down; memory of a go-live fades fast and the lessons are the most valuable output.

8. Deliverables Summary — Epic Go-Live & Command Center Endpoint Readiness Playbook

This solution ships a complete, ready-to-run package: the documentation set, five PowerShell automation scripts, and a hero image. Everything is white-label and reusable across waves, sites, and upgrades.

Documentation

#	Document	Purpose
1	`01-solution-overview.md`	What the playbook is, the problem it solves, scope, phase model, and the named artifacts it produces.
2	`02-requirements.md`	Solution-specific requirements R1–R12, each with Description / Why / Verify / Implement-if-missing.
3	`global-requirements-healthcare-solutions.md`	Category-wide prerequisites (tenant/roles, licensing, HIPAA/HITRUST + change control, maintenance windows, network, test device, rollback).
4	`03-implementation.md`	The five-phase operational walkthrough (Assess → Prepare/Remediate → Validate → Command Center → Stabilize) with full console paths and the triage runbook.
5	`05-testing.md`	Validation sweep, readiness scorecard, functional spot-tests, remediation rehearsal, command-center dry run, and gate/exit-criteria validation.
6	`06-troubleshooting.md`	Deep diagnostics for the common day-1 endpoint problems and tooling issues.
7	`07-best-practices.md`	Accumulated discipline across readiness, change safety, command center, comms, compliance, and handover.

#	Document	Purpose
8	`08-deliverables-summary.md`	This document.
—	`executive-summary.md`	One-page executive framing for program and clinical leadership.

PowerShell scripts

All scripts use Microsoft Graph where appropriate, carry a standard header block, and are **read-only by default**. The `.NOTES` block on each is neutral: *"Healthcare Solutions — read-only."* State-changing actions require an explicit switch and run only inside an approved maintenance window against the readiness device group.

#	Script	What it does	Default mode
1	`EPICGOLIVE-Discovery.ps1`	Fleet readiness snapshot — per device: enrollment, compliance, OS build, detected Epic client version, Imprivata agent presence, last check-in. Serves as the Assess baseline and the rollback before-state.	Read-only
2	`EPICGOLIVE-Validation.ps1`	Per-device go/no-go validation against the readiness checklist; returns **PASS/FAIL** with reasons per check.	Read-only
3	`EPICGOLIVE-Report.ps1`	Rolls validation results into the **Readiness Scorecard – Clinical Fleet – Validation** and exports to CSV (per-device + roll-ups; no ePHI).	Read-only

#	Script	What it does	Default mode
4	`EPICGOLIVE-Remediation.ps1`	Safe, supported remediation of enrollment/compliance gaps. Previews actions by default; changes state only with `-Apply`.	`-WhatIf` (read-only)
5	`EPICGOLIVE-Set-ReadinessGroup.ps1`	Manages membership of the **Readiness Device Group – Clinical Fleet – Validation** from the in-scope inventory. Previews intended membership by default.	`-WhatIf`

Visual asset

File	Purpose
`EPICGOLIVE-hero.png`	Hero image for the solution listing.

Named artifacts the playbook produces (operating outputs)

These are produced *by running the playbook* in a customer environment (not shipped pre-filled), each named `<Purpose> – <Scope> – <Stage>`:

- **Endpoint Readiness Checklist – Clinical Fleet – Pre-Go-Live**
- **Go/No-Go Gate – Endpoint Domain – Activation**
- **Readiness Scorecard – Clinical Fleet – Validation** (output of `EPICGOLIVE-Report.ps1`)
- **Command Center Roster – Endpoint Support – Hypercare**
- **Rapid-Fix Triage Runbook – Day-1 Endpoint – Hypercare**
- **Comms & Escalation Plan – Endpoint Domain – Activation**
- **Hypercare Exit Criteria – Endpoint Domain – Handover**
- **Readiness Device Group – Clinical Fleet – Validation** (managed by `EPICGOLIVE-Set-ReadinessGroup.ps1`)

How the pieces fit

1. **Assess** with `EPICGOLIVE-Discovery.ps1` and create the **Readiness Device Group** via `EPICGOLIVE-Set-ReadinessGroup.ps1 -WhatIf`.

2. **Remediate** gaps with `EPICGOLIVE-Remediation.ps1 -Apply` (tested first, in a window).

3. **Validate** with `EPICGOLIVE-Validation.ps1` and produce the **Readiness Scorecard** with `EPICGOLIVE-Report.ps1`.

4. **Decide** at the **Go/No-Go Gate**.

5. **Run hypercare** with the **Roster**, **Triage Runbook**, and **Comms & Escalation Plan**.

6. **Hand over** on the **Hypercare Exit Criteria**.

Related solutions (deep configuration)

• **Epic client deployment & standardization solution** — Hyperdrive/Hyperspace packaging, versioning, deployment rings.

• **Imprivata tap-and-go / SSO solution** — OneSign agent, enrollment, policy.

• **Clinical workstation standardization solution** — master image, baseline, hardware standards.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

EPICGOLIVE-Discovery.ps1

EPICGOLIVE-Remediation.ps1

EPICGOLIVE-Report.ps1

EPICGOLIVE-Set-ReadinessGroup.ps1

EPICGOLIVE-Validation.ps1